Homework 3: Language Modeling

Chuttarsing Emeryc

(a) Tokenizer (1pt)

Briefly describe what BPE does and why a subword tokenizer like BPE is preferable...

BPE is an algorithm to create a new group of token by recognizing repetitive pattern. It is better than just using character level or full word tokenization because in both case, they fail to detect pattern : character level don't achieve to detect group of character for instance (which means words) while using full word tokenization fails to detect group of words or grammatical patterns like -ing or -ed. Not using BPE is working with blind token.

- **Vocabulary Size:** 3000

(b) Dataset Construction (1pt)

Report the total number of tokens in your training and validation sets, and the number of training samples after chunking.

- **Total tokens (Training):** $\approx$ 40,681,695
- **Total tokens (Validation):** $\approx$ 1,007,308
- **Total training samples (after chunking):** 40,681,695

(c) Model Architecture (2pts)

Provide a brief description of the architecture showing how data flows from input token IDs to output logits.

Input token ID sequences are first converted into dense vectors via a Token Embedding layer, to which sinusoidal Positional Encoding is added to integrate the concept of sequence order. These representations then flow through a stack of N successive Transformer blocks, where each block refines the information by applying a "Pre-norm" layer normalization, a Causal Multi-Head Self-Attention mechanism (to process context while masking future words), and a position-wise Feed-Forward network, all supported and stabilized by residual connections. Upon exiting these blocks, the data stream passes through a final Layer Norm before an output projection layer—whose weights are tied to the initial Token Embedding—transforms these vectors into the final output logits corresponding to the vocabulary size.

(d) Training (3pts)

Report the plots, validation loss/perplexity, training time. Does your initial loss match what you would expect from random predictions?

The expected loss is supposed to be higher at the beginning of the training.

- **Total training time:** 1h



Figure 1: Training Loss and Validation Perplexity vs. Gradient Step

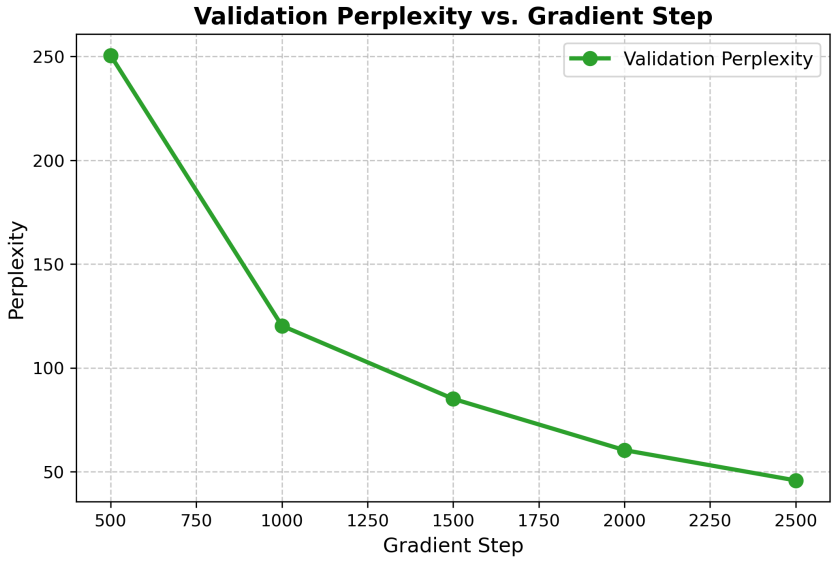


Figure 2: Training Loss and Validation Perplexity vs. Gradient Step

(e) Generation and Analysis (3pts)

Generate at least 5 samples using different prompts. Comment on the quality.

Generated Samples:

- **Prompt:** *"Once upon a time"*
  Continuation: `, blew tw en see ished Timmy taste confused led gre ins Don That law iness ess bet spe ds wi.`

- **Prompt:** *"The little dog"*
  Continuation: `, urn continued faster spill really hugged jumped har k Lily soap`

`rainbow boys Bird Now serious c children cars plane bad walked Jimmy S ipp fla collect off fairy search care bring ray Don mo Max free paw th thoughtful ry naughty cloth ined aw port ined mar fe nest`

- **Prompt:** *"One day"*
  Continuation: `, a girl named smart terrible park trouble al ummy yal yal yal yal yal yal yal yal yal yal yal yal yal yal yal yal yal yal yal yal yal yal yal yal yal last balance helped too cu med twist over collar rare large Bear zoo pillow pretended turns ess vide proud looking`

- **Prompt:** *"The little cat"*
  Continuation: `squash to deer some ain funny do talked - lly frightened Mia batter tough covered like Mum hugs rare look pract sunflower drink lie jo frustr aunt mask Whiskers ali mail anut anut anut anut anut anut anut anut anut anut anut anut uni garage a After locked named am`

- **Prompt:** *"Tim found a red box"*
  Continuation: `pack idea A frightened Jerry ." arrived door most gif whole on an can celebr bite forest meet ing ore It edy corner Max fear ther rod ug spent chased spoiled cle bar instead mark sun ies y around shapes jam ha getting tiny wagon Sally frightened coat gray S`

Analysis:
The model successfully generates thematic vocabulary typical of the TinyStories dataset—such as common children's names, animals, and concepts—but it currently lacks any structural understanding of language. The outputs are not grammatically correct sentences but rather incoherent "word salads" that frequently include broken sub-word fragments. Furthermore, there is absolutely no narrative coherence or plot progression; instead, the model suffers from severe token degeneration, getting stuck in endless repetition loops (e.g., repeating "yal" or "anut"). Ultimately, while the model has memorized which words belong in a children's story, it has not yet learned how to construct meaningful sentences or narrative arcs.

(f) Exploring the Embedding Space (1pt)

Do you find any interesting occurrences in the embedded space? Give examples.

Results:

- Closest word to `'dog'` : `['dog', 'boy', 'cat', 'named', 'hairy', 'girl']`
- Results of `'puppy' - 'dog' + 'cat'` : `['puppy', 'cat', 'fat', 'bunny', 'gray', 'bird']`

Analysis:
When exploring the embedding space, I noticed semantic clustering, even though the model hasn't been trained for very long. For the single-word example, I checked the nearest neighbors for "dog" and got words like "cat". This shows the model is already figuring out that animal words belong together based on their context. For the token arithmetic, "puppy" - "dog" + "cat" got "puppy" as one of the closest vectors! Even though the model still struggles to generate coherent sentences, these results show that the embedding layer is definitely starting to pick up on basic word relationships and categories from the dataset.

(g) Experiment: Effect of Vocabulary Size (2pts)

How does vocabulary size affect both the loss value and the qualitative readability of generated text?

Increasing the vocabulary size (to 5,000) improves the qualitative readability of the generated text. With a smaller vocabulary, the tokenizer is forced to break words down into much smaller sub-word fragments or even individual characters. This makes it much harder for the model to learn correct spelling and word boundaries, leading to more fragmented "word salads." With a larger vocabulary, tokens represent larger, more meaningful chunks of text (often whole words), which drastically reduces spelling errors and makes the generated sentences look much more like natural English, even if the underlying grammar is still weak.